



IUT DE SAINT DIE DES VOSGES

2025/2026

Rapport de stage

Digital ALL Things

**AUTOMATISATION DES PROCESSUS INFORMATIQUES D'UN PROJET DE
DÉVELOPPEMENT.**



Rayane ABABSA

mars-avril 2026

Remerciements

Je tiens tout d'abord à remercier L'IUT de Saint DIE des Vosges, et particulièrement Mr Lahcène Harbouche pour m'avoir fait confiance. L'écoute et l'accompagnement dont j'ai bénéficié m'ont permis de trouver rapidement un stage, dans le but d'affiner mon projet professionnel et d'obtenir à la fin de l'année scolaire mon dut informatique

Je pense également à Mr Lahcène Harbouche qui a cru en mon potentiel et m'a accueilli(e) au sein de son entreprise. À ce titre, je souhaiterais le remercier tout particulièrement, qui m'a épaulé(e) et conseillé(e) et qui m'a surtout transmis son expertise dans le domaine du cloud.

Ce stage m'a permis d'affiner certaines pistes pour bâtir mon projet personnel qui est de pouvoir intégrer le marché du travail dans la cybersécurité et constitue l'aboutissement de mon année scolaire 2025/2026 .

Je n'oublie pas non plus mes proches qui m'ont sans cesse soutenu dans l'élaboration de mon projet professionnel et m'ont aidé à chaque étape de ce rapport de stage.

Remerciements spéciaux à mon relecteur et correcteur qui a contribué, grâce à ses conseils et recommandations, à l'élaboration et au bon déroulé de mon rapport de stage

Table des matières

Introduction

présentation de l'entreprise

Étude

a. Analyse des besoins et présentation des missions du stage

b. Présentation des solutions techniques possibles et justification de leur emploi

c. Méthodologie de travail (ressources, calendrier, organisation, etc.)

d. Mise en œuvre de la/des solutions retenues et résultats

Conclusion et perspectives personnelles

Bibliographie

Dictionnaire de mots techniques

Annexes

Introduction

Le stage s'inscrit dans le cadre du sujet automatisation des processus informatiques d'un projet de développement. L'objectif principal confié était de concevoir et de développer une solution permettant d'industrialiser la communication entre une interface no code (bubble) et un backend serverless sur aws tout en automatisant les traitements de données, le stockage et le déploiement afin que la structure soit portable.

L'entreprise Digit all things est une petite boîte spécialisée en transformation digitale en intégrant des solutions digitales à des processus non informatisés mais aussi en répondant à la demande informatique du client. Au cours de ce stage dans le service IT de l'entreprise, j'ai pu m'intéresser au métier du Full Stack cloud.

L'entreprise située à Esch sur Alzette, créée en 2017, est spécialisée dans la transformation digitale des processus. J'ai été soutenu par, Lahcène qui est le CEO de l'entreprise avec qui j'ai pu me former dans d'excellentes conditions.

Durant ces deux mois de stage (Du 02 mars 2026 au 24 avril 2026), j'ai pu exercer quelques métiers assez importants pour mon développement comme le développeur Backend DevOps administrateur de base de données mais aussi tout ce qui est relation client comme un vrai commercial en passant par la fonction de rédaction.

Cette expérience a été pour moi une opportunité de percevoir comment une entreprise spécialisée en se développer et de relever avec elle le défi à laquelle elle était confrontée et notamment face à la question de l'automatisation du processus de déploiement d'une architecture.

L'élaboration de ce rapport a pour principale source la pratique journalière des missions qui m'étaient affectées, mise en parallèle avec les enseignements théoriques de ma formation.

Je vais vous présenter dans ce rapport le déroulement du stage en commençant par une présentation rapide de l'entreprise puis par l'étude qui se compose de l'analyse des besoins et de la présentation des missions de stage, présentation des solutions techniques et leur justification de leur emploi, méthodologie de travail en passant par la mise en œuvre des solutions retenues et de leur finalité enfin nous concluons et abordons des perspectives personnelles.

présentation de l'entreprise

Fondée en janvier 2017 par lahcène harbouche, l'entreprise est née de la volonté de répondre à un besoin d'informatisation des entreprises dans leur processus non informatisé vers une structure informatique et il se développa au début seul mais ensuite il se fait accompagner par une petite équipe qui est à ses côtés.

fonction	nom
CEO de l'entreprise	lahcène harbouche

et pour le reste me direz-vous; la réponse est simple c'est que je l'ai vu une fois et après cela il était que sur le terrain donc j'ai pas eu le temps de faire connaissance avec eux.

Études

L'étape de l'étude constitue le pivot central de ce projet. C'est le moment où les aspirations théoriques de l'entreprise rencontrent les contraintes techniques du cloud.

Analyse des besoins et présentation des missions du stage

Avant de pouvoir commencer, il faut définir les exigences non fonctionnelles qui garantissent la pérennité du processus. J'ai identifié 4 piliers fondamentaux afin de mener le projet à bon terme.

Premier point le plus important la sécurité donc une approche "zero trust" :

Dans un environnement où les données transitent entre une plateforme SaaS (Bubble) et un environnement cloud privé (AWS), la sécurité ne pouvait être une option. L'analyse a révélé la nécessité d'une authentification robuste à chaque point d'entrée de l'API. J'ai opté pour un chiffrement systématique des données en transit via TLS 1.2 pour interdire toute interception malveillante.

Au niveau de la gestion des accès, le principe du moindre privilège a été appliqué rigoureusement au rôle IAM. Chaque fonction lambda ne possède que les droits strictement nécessaires, par exemple la fonction d'écriture peut pousser un objet dans un bucket S3 mais n'a aucun droit de lecture ou de suppression sur le reste de

l'infrastructure. Enfin, pour garantir l'intégrité historique j'ai exigé un archivage immuable des pilotes empêchant toute modification à posteriori des traces transactionnelles.

-La performance:

Pour l'utilisateur final sur Bubble. La frontière entre le front end et le back end doit être invisible. L'analyse des besoins a fixé un seuil d'intensité minimal. Latence. Extrêmement strict. Les opérations critiques de synchronisation doivent être traitées en moins de 1000 millisecondes. Ce défi impose une optimisation fine du Code Python et une configuration Millimétrée des capacités de lecture Slash écriture de Dynamodb. Pour éviter les goulots d'étranglement lors des pics de charge.

-La traçabilité :

Afin de répondre aux exigences d'audit et de débogage. La traçabilité a été pensée à double échelle. D'un côté. Une traçabilité instantanée via AWS Cloudwatch. Permettant de surveiller en temps réel la santé des fonctions lambda. Et de lever des alertes en cas d'erreur. De l'autre , Une traçabilité historique via l'archivage systématique de chaque requête JSON Dans un bucket Amazon S3 structuré. Cette méthode permet de rejouer n'importe quelle transaction en cas de litige ou de panne logicielle.

-L'automatisation:

Le dernier besoin identifié ? Et sans doute le plus complexe. Et celui de l'indépendance vis-à-vis de la configuration manuelle. La solution doit être reproductible à l'infini. Cela implique la mise en place d'un pipeline de déploiement continu. Cidc. Capable d'effectuer un rollback automatique. Retour en arrière ? Si une mise à jour ne passe pas les tests de santé. Assurant ainsi une disponibilité constante du service.

Présentation des solutions techniques possibles et justification de leur emploi

Une fois ces besoins traduits en exigences techniques, mon stage s'est articulé autour d'un parcours de développement complet, allant de la conception abstraite à la documentation finale.

Ma première mission a été celle de l'architecture. j'ai dû produire un schéma d'architecture globale détaillant les flux de données. Cette étape a été doublée d'une mission de gestion de projet classique (rédaction du cahier des charges, établissement d'un planning prévisionnel et surtout une estimation des coûts via l'outil AWS Pricing calculator. Cette dimension financière est cruciale en entreprise pour valider la fiabilité économique du passage au Serverless.

Développement de la logique métier (le cœur du projet)

Le cœur de ma mission technique consistait en la rédaction des fonctions Lambda en Python. En utilisant le SDK boto3 (bibliothèque spéciale en python), j'ai programmé la logique permettant de passer les requêtes JSON entrantes, de valider les schémas de données et de router l'information vers les services de stockage. Ce travail a exigé une attention particulière à la gestion des erreurs pour s'assurer que l'application ne plante jamais silencieusement

Création du pont de communication (API)

Pour que bubble puisse déclencher ces fonctions, j'ai mis en place une API REST via l'api Gateway de aws. Cette mission inclut la configuration délicate des politiques CORS. Sans une configuration précise des headers côté AWS et côté bubble, la communication navigateur serait bloquée par la sécurité standard du web. j'ai donc dû harmoniser ses deux environnements pour permettre un dialogue fluide.

Modélisation et persistance des données

La mission s'est poursuivie par la modélisation des bases de données. j'ai structuré une table dynamoDB et configuré un bucket Amazon S3 point l'enjeu ici était de définir une structure clé de partition efficace sur dynamoDB pour optimiser les performances de recherche.

Industrialisation via l'infrastructure :

Afin de rendre le projet portable j'ai rédigé un template Sam. Ce Modèle Yaml décrit l'intégralité de l'infrastructure comme du code. grâce à cela, le déploiement ne se fait plus par clic sur une console mais par une PAP line automatisé garantissant qu'aucune erreur humaine depuis ce qu'on ronfle environnement de production. Pour cela que dans l'infrastructure pour faciliter le déploiement et j'ai rajouté un fichier. bash, son rôle consiste à déployer l'architecture qui est contenu dans un fichier zip ou le client le pourra le dézipper et exécuter le fichier .bash.

Validation et documentation technique

La dernière phase de ma mission a été consacrée au test d'intégration. j'ai vérifié la chaîne complète du clic bubble jusqu'à l'écriture finale dans S3 en passant par l'autorisation IAM point le stage s'est conclu par rédaction d'un document technique final exhaustive, inclure les configurations IAM fines, la structure d'archivage S3 et les rapports de performance permettant à l'entreprise de reprendre le projet en toute autonomie.

Justification de leur emploi

Pour obtenir la réussite de cet architecture cela a été un défi au niveau des choix de logiciel et d'application je tiens à préciser que tous ces applications ont été imposées par le tuteur de stage. Ce n'est pas pour autant que je vais pas justifier les choix de l'entreprise, pourquoi ce logiciel ou cette application?

premièrement mon tuteur étant un spécialiste dans le cloud et n'ayant qu'un peu de compétences dans la programmation il s'est tourné vers bubble pour faire son site car cela est un application de code donc facilement maniable une deuxième option

on peut obtenir les avantages bubble gratuitement et la configuration de son API et facilement configurable et bien sûr bubble est une application SaaS et donc très sécurisé avec une base de données intégré à bubble. même s' il existe des solutions autre que bubble le choix a été quand même bubble.

Au niveau du backend le choix a été évidemment AWS qui est l'application de Amazon au niveau du back end qui est très très performant Quasi presque toutes les grandes entreprises américaines l'utilise bon extrêmement polyvalent ,la politique de AWS qui est de payer ce qu'on consomme et inverse de pas payer quand on ne consomme pas service accessible 24 heures sur 24 7 jours sur 7 avec des bases de serveur un peu partout dans le monde. Donc comme c'est un outil extrêmement répandu j'ai dû l'utiliser même si après j'ai dû faire une formation pour pouvoir utiliser AWS à son plein potentiel.

Puis au niveau de ses composants logiquement je me suis concentré sur le buckets s3, le lambda, la base de donnée dynamodb, les rôles iam, les api ,aws sam, cli et le sdk. Et tout ceci c'est ce qui gère toute l'infrastructure finale et leur configuration et leur optimisation sont cruciaux Tout comme leur sécurité qui est très essentiel.

Méthodologie de travail (ressources, calendrier, organisation, etc.)

Le succès de ce projet d'architecture hybride repose sur une approche itérative et rugueuse étalée sur 8 semaines entre “2 mois” Cette section détaille le cheminement logique de la montée en compétence initiale jusqu'à la livraison de la documentation finale.

Phase 1 :

La première étape de mon stage a été consacrée à une immersion technologique point bien que possédant des bases de programmation l'écosystème de AWS et la spécialité d'une école industrielle exigent une mise à niveau profonde.

Formation AWS : j'ai suivi un cursus intensif sur tous les services de AWS avec des cours de théorie jusqu'à de la pratique Enfin à la fin de chaque apprentissage j'ai été testé sur le niveau de maîtrise de secours ou du module. Ici l'enjeu était de comprendre qu'on ne gère plus de serveur mais des événements donc s'adapter à la clientèle.

aws educate

Cours et ateliers Emplois Communauté de talents émergents Rayane ABABSA

Filtres

- Caractéristiques du cours
- Compétences
- Niveau
- Durée
- Langue

Rechercher des cours à l'aide de mots clés

En cours

Resume Creation

0%

Foundational | 1 hour(s)

Professional skills

Introduction to Cloud 101

0%

Foundational | 3 hour(s)

Cloud Computing

Mise en route

Vous débutez avec le cloud computing ? Commencez ici.

Getting Started with Networking

Foundational | 2 hour(s)

Network & Infrastructure

Getting Started with Databases

Foundational | 2 hour(s)

Cloud Computing

Getting Started with Cloud Operations

Foundational | 2 hour(s)

Cloud Computing

Getting Started with Security

Foundational | 2 hour(s)

Security

2/3

Explorer

En savoir plus sur AWS

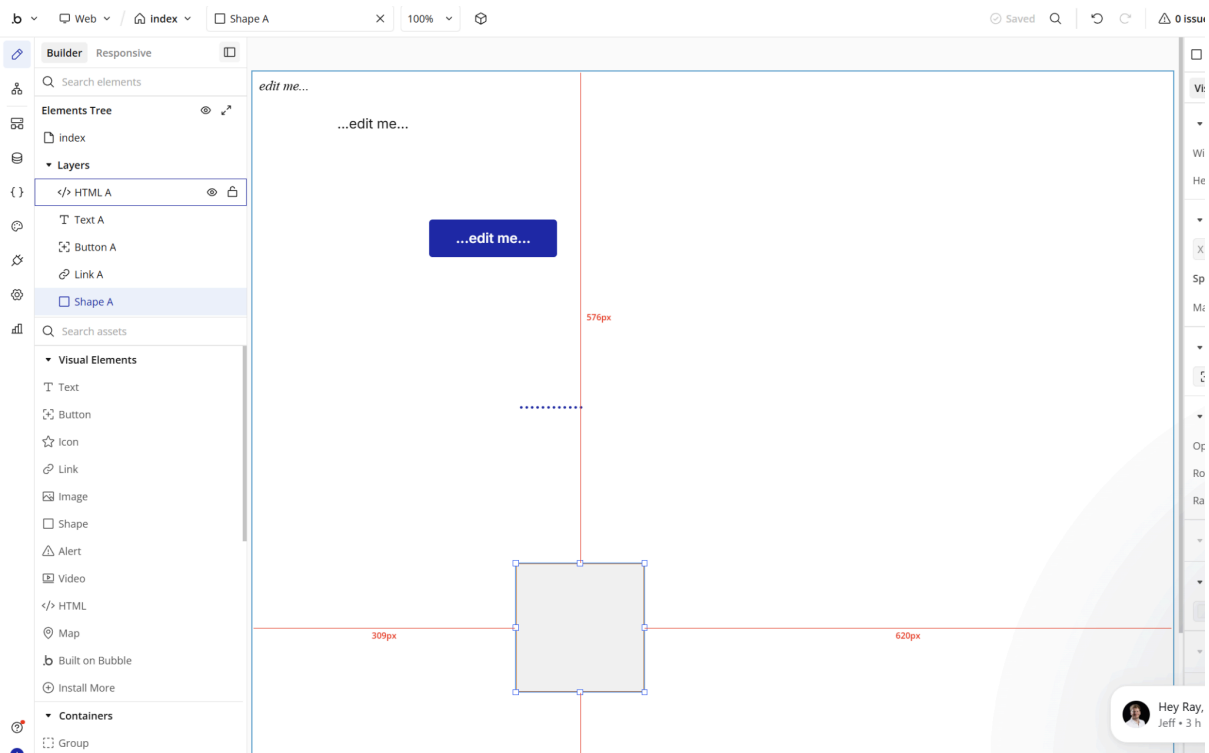
Badges AWS Educate (8)

Les badges que vous avez obtenus en suivant les cours AWS Educate s'affichent automatiquement ici.



mise à niveau Python : Sous la direction de mon tuteur, j'ai approfondi l'usage de python dans le contexte Cloud l'accent a été mis sur la bibliothèque Boto 3, le STK officiel d'Amazon, indispensable pour piloter les ressources AWS par le code.

Au niveau de bubble j'ai explorer des capacités avancées de l'API connector (le plugin) notamment la gestion des appels asynchrones et des réponses JSON complexes. Il a fallu aussi intégrer à l'application de bubble des fonctionnalités des API J'ai pu trouver sur internet. Même si Bubble est un no code la configuration des différentes interactions et visuelles pour le client a été un défi de logique.



Phase 2 : Analyse du cahier des charges (semaine 3). Une fois le socle technique maîtrisé, j'ai procédé au recueil des besoins auprès de la direction de Digital All Things.

Recueil du besoin : l'objectif était de créer une solution de synchronisation de données bidirectionnelles capable de supporter une montée en charge sans intervention humaine. Définition du périmètre : Nous devons délimiter le projet à la création d'un pipeline sécurisé (Front-end Bubble, API Gateway, Lambda, DynamoDB, S3). Contraintes techniques : La solution devra répondre à trois impératifs minimum : sécurité maximale et portabilité totale.

Phase 3 conception de l'architecture semaine 4

La phase de conception a permis de transformer les besoins en schéma technique exploitable

Film architecture j'ai rien dit une cartographie des flux de données montrant comment chaque service interagissent modélisation des données MCD MLD nous pour dynamo DP nous avons opté pour une approche non SQL contrairement ou est-ce que les traditionnel la conception s'est fait par accès j'ai défini la partition clé de Lady utilisateur pour garantir des performances optimales définition des rôles c'est ici qu'Internet la sécurité j'ai réalisé les politiques d'accès granulaire par exemple lambda de traitement a reçu le droit dynamo dB mais c'est vu refuser dynamo dilate table appliquant ainsi le principe du moindre privilège

Phase 4 développement applicatif le développement a été scindé en deux parties le code métier et l'automatisation de l'infrastructure

Fonction lambda moléculaire au lieu de créer un code monovitique chez segmenté la logique en mode Jupiter réutilisable validation formatage insertion cela facilite là maintenant c'est les tests des cimetières logo avant tout déploiement j'ai utilisé des simulateurs d'événements pour tester mes creep bidon en local évident ainsi d'aller au retour inutile dans le cloud et optimiser le temps de développement.

Phase 5 : Infrastructure as Code (IaC) et CI/CD. Semaine six. Pour garantir la reproductibilité réclamée par l'entreprise, j'ai banni toute configuration manuelle. Rédaction du template YAML : J'ai décrit l'intégralité de l'infrastructure. Quand YAML est devenu la source de vérité du projet. Chaîne CI/CD : J'ai mis en place des scripts de packaging et le déploiement via la console SAM. Désormais, une simple commande ``sam deploy`` permet de mettre à jour l'intégralité du système, de l'API Gateway aux bases de données, avec une gestion automatique des versions et des rollbacks facilités. Tests d'intégration et validation (semaine 7). La phase de test a consisté à vérifier la chaîne de communication de bout en bout : connexion Bubble AWS, vérification de la bonne transmission des headers JSON via l'API Gateway, vérification via CloudWatch. J'ai analysé les logs en temps réel pour traquer d'éventuels déplacements de mémoire ou des erreurs de timeout sur les fonctions Lambda. Test de charge : simulation de plusieurs requêtes simultanées pour observer le comportement de montée en charge, l'auto scaling de DynamoDB. Rapports finaux et clôture : Le stage s'est achevé par l'acquisition et la livraison des livrables clients (documentation technique, rédaction d'un guide d'utilisation incluant des captures des configurations, des extraits de code, le succès et les schémas finaux). Estimation des coûts : comme dit précédemment, cela a été fait avec AWS Pricing Calculator, montrant la rentabilité de la solution et des services par rapport à un hébergeur classique.

Outils et ressources utiliser

Bubble

Et AWS dans AWS on a les lambda qui utilisent du python (choix du maître du stage mais sinon on peut le faire en plusieurs langues de programmation) ensuite via le dynamodb j'ai utilisé du nosql et aussi du yaml via le cloud formation

Mise en œuvre de la/des solutions retenues et résultats

Cette section détaille la phase de construction technique de l'architecture. La mise en œuvre a été guidée par un objectif de fluidité : faire en sorte que l'utilisateur sur Bubble ne ressente jamais la complexité du traitement qui s'opère en arrière-plan sur AWS. Ah, le data jour, de l'événement au stockage immuable. Le cœur du système repose sur un flux de données automatisées et sécurisé. Chaque action sur l'interface Bubble déclenche une réaction en chaîne sur AWS. L'émission Bubble. L'application front-end génère un pilote JSON contenant les informations saisies par l'utilisateur. 2. Le transit API Gateway. La requête est réceptionnée par l'API Gateway qui valide immédiatement si le format est correct et si l'appelant est

autorisé. 3. Le traitement. L'angle à Python. C'est ici que réside l'intelligence. J'ai développé un script Python qui exécute trois actions critiques : nettoyage et validation. Le script vérifie la cohérence des données (type, longueur des chaînes) pour éviter toute corruption de la base. Persistance transactionnelle. La donnée propre est écrite dans DynamoDB. Archivage immuable. Parallèlement, le JSON brut est encapsulé et stocké dans un bucket S3. Cette double écriture garantit qu'en cas d'erreur de logique plus tard, nous possédons toujours la preuve originale du message envoyé. Des configurations du pont. Bubble API Connector et API Gateway. La mise en place de la communication a été l'un des défis techniques les plus stimulants. Il a fallu synchroniser deux mondes aux philosophies différentes : l'interface Bubble, le plugin API Connector. Pour permettre à Bubble de sortir de son écosystème, j'ai configuré le plugin API Connector.

Initialisation des méthodes. Bien que l'on utilise souvent la méthode GET pour la récupération de données, j'ai principalement configuré des méthodes POST pour l'envoi de données vers AWS. Headers d'authentification. J'ai mis en place des clés d'accès sécurisées (API Keys) injectées dans les headers pour que seule notre application Bubble puisse réveiller l'infrastructure AWS. Le récepteur AWS API Gateway. Côté Amazon, la configuration a été faite via API Gateway pour créer une interface REST. Ressources et méthodes. J'ai défini une structure de ressources hiérarchiques avec des méthodes spécifiques (POST). Intégration proxy. L'activation de l'intégration proxy a permis de transmettre l'intégralité du contexte de la requête à la fonction Lambda, offrant une flexibilité totale lors de la phase de développement Python. Développement et déploiement. L'approche SDK est IAC. La logique Python. Boto 3. Le script Python a été structuré de manière rigoureuse.

```
import json
import boto3
import os
import urllib3 # Pour appeler l'API Bubble

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table(os.environ['TABLE_NAME'])
http = urllib3.PoolManager()

def handler(event, context):
    # 1. LOGIQUE DE RÉCUPÉRATION BUBBLE
    # On imagine que tu veux déclencher une synchro via un POST
    if event.get('httpMethod') == 'POST':
        # Remplace par ton URL Bubble réelle
        BUBBLE_API_URL = "https://ton-app.bubbleapps.io/api/1.1"
        headers = {"Authorization": "Bearer "
        }

        try:
            r = http.request('GET', BUBBLE_API_URL, headers=headers)
            bubble_data = json.loads(r.data.decode('utf-8'))

            # 2. STOCKAGE DANS DYNAMODB
            # On boucle sur les résultats de Bubble (ajuste selon la structure JSON de Bubble)
            results = bubble_data.get('response', {}).get('results', [])
            for item in results:
                table.put_item(Item={
                    'userId': item.get('_id'), # ID unique Bubble
                    'nom': item.get('nom_field'),
                    'donnees': json.dumps(item)
                })
```

```
def handler(event, context):
    if event.get('httpMethod') == 'POST':
        try:

            # 2. STOCKAGE DANS DYNAMODB
            # On boucle sur les résultats de Bubble (ajuste selon la structure JSON de Bubble)
            results = bubble_data.get('response', {}).get('results', [])
            for item in results:
                table.put_item(Item={
                    'userId': item.get('_id'), # ID unique Bubble
                    'nom': item.get('nom_field'),
                    'donnees': json.dumps(item)
                })

            return response(200, {"message": f"{len(results)} éléments synchronisés"})
        except Exception as e:
            return response(500, {"error": str(e)})

    # 3. LOGIQUE DE LECTURE (GET)
    elif event.get('httpMethod') == 'GET':
        items = table.scan().get('Items', [])
        return response(200, items)

    return response(405, {"message": "Méthode non autorisée"})

def response(status, body):
    return {
        'statusCode': status,
        'headers': {'Content-Type': 'application/json', 'Access-Control-Allow-Origin': '*'},
        'body': json.dumps(body)
    }
```

Page d'accueil de la console

Informations

[Réinitialiser la mise en page par défaut](#)
[+ Ajouter des widgets](#)

Recent visited Informations

CloudWatch

S3

Lambda

CloudShell

Gestion de la facturation et des coûts

DynamoDB

API Gateway

CloudFormation

Certificate Manager

Infrastructure Composer

IAM

Systems Manager

EC2

[Afficher tous les services](#)

Applications (0) Informations

Région : Europe (Paris)

Sélectionner la région
eu-west-3 (Région actuelle)

Rechercher des applications

Nom	Description	Région	Compte d
Aucune application			
Commencez par créer une application.			

Créer une application

Accédez à myApplications

Bienvenue sur AWS Informations

Démarrage avec AWS

Découvrez les bases et trouvez des informations précieuses pour tirer le

AWS Health Informations

Problèmes en cours

0 Notifications des 7 derniers jours

Modifications prévues

Coût et utilisation Informations

Mettre à niveau le plan

Les crédits couvrent les coûts de votre plan gratuit. Votre accès aux services AWS prendra fin lorsque les crédits seront épuisés ou quand la période gratuite prendra fin.

Crédits restants | Jours restants

```
~ $ ls
mon-projet-final
~ $ cd mon-projet-final
mon-projet-final $
mon-projet-final $ ls
architecture.yaml --capabilities deployment.sh deploy-expert.zip lambda --output-template-file packaged.yaml --s3-bucket --stack-name --template-file
mon-projet-final $
```

DynamoDB

Tableau de bord

Tables

Explorer les éléments

Éditeur PartiQL

Sauvegardes

Exportations vers S3

Importations depuis S3

Intégrations

Capacité réservée

Paramètres

Tables (2)

Filter par identification

N'importe quelle clé de balise

Filter par valeur de balise

N'importe quelle valeur de balise

Rechercher des tables

mon-projet-final-MaTableExpert-O4YTNABTHOGF

MonArchitectureExpert-MaTableExpert-13E6RO9VF6U98

mon-projet-final-MaTableExpert-O4YTNABTHOGF

Autopreview

Afficher les détails de la table

Analyser ou interroger des éléments

infos

Analyser

Interrogation

Sélectionner une table ou un index

Table - mon-projet-final-MaTableExpert-O4YTNABTHOGF

Sélectionner la projection d'attributs

Tous les attributs

Filtres - facultatif

Exécuter

Réinitialiser

Terminé

Éléments retournés : 3

Éléments analysés : 3

Efficacité : 100%

RCU consommées : 2

Table : mon-projet-final-MaTableExpert-O4YTNABTHOGF – Éléments retournés (3)

Actions

Créer un élément

L'analyse a commencé le mai 03, 2026, 20:43:38

userId (Chaîne)

data

status

1773234437785x859...

{ "user_sign...

synced

1775113300538x739...

{ "id": "177...

synced

1775233581265x188...

{ "nom": "tes...

synced

Ressources IAM

Ressources de ce compte AWS

Groupes de personnes

0

Personnes

1

Rôles

7

Politiques

1

Fournisseurs d'identité

0

retour

Type here...

	1. FC Köln
	AC Pisa 1909
	Deportivo Alavés
	Angers SCO
	Arsenal FC
	Aston Villa FC
	Atalanta BC
	Athletic Club
	Club Atlético de Madrid
	FC Augsburg
	AJ Auxerre
	FC Barcelona
	FC Bayern München
	Bologna FC 1909
	AFC Bournemouth
	SV Werder Bremen
	Brentford FC
	Stade Brestois 29
	Brighton & Hove Albion FC
	Burnley FC
	Cagliari Calcio
	RC Celta de Vigo
	Chelsea FC
	Como 1907
	US Cremonese
	Crystal Palace FC

The code is higher than your real design as the debugger is visible and displays a white area.

retour	Type here...		
	1. FC Köln	réseaux twitter	actualité
	AC Pisa	réseaux twitter	actualité
	Alavés	réseaux twitter	actualité
	Angers SCO	réseaux twitter	actualité
	Arsenal	réseaux twitter	actualité
	Aston Villa	réseaux twitter	actualité
	Atalanta	réseaux twitter	actualité
	Athletic	réseaux twitter	actualité
	Atleti	réseaux twitter	actualité
	Augsburg	réseaux twitter	actualité
	Auxerre	réseaux twitter	actualité
	Barça	réseaux twitter	actualité
	Bayern	réseaux twitter	actualité
	Bologna	réseaux twitter	actualité
	Bournemouth	réseaux twitter	actualité
	Bremen	réseaux twitter	actualité
	Brentford	réseaux twitter	actualité
	Brest	réseaux twitter	actualité
	Brighton Hove	réseaux twitter	actualité
	Burnley	réseaux twitter	actualité
	Cagliari	réseaux twitter	actualité
	Celta	réseaux twitter	actualité
	Chelsea	réseaux twitter	actualité
	Como 1907	réseaux twitter	actualité
	Cremonese	réseaux twitter	actualité
	Crystal Palace	réseaux twitter	actualité

Compartiments

Compartiments à usage général

Toutes les régions AWS

Compartiments de répertoires

Compartiments à usage général (5) [Infos](#)



 Copier l'ARN

Vider

Supprimer

Créer un compartiment

Les compartiments sont des conteneurs pour les données stockées dans S3.

Rechercher des compartiments par nom

< 1 >



	Nom	Région AWS	Date de création
☐	cf-templates-1rxn47dszfoi-eu-west-3	Europe (Paris) eu-west-3	03 Apr 2026 12:13:53 PM CEST
☐	deploy-expert-177574772	Europe (Paris) eu-west-3	09 Apr 2026 05:17:19 PM CEST
☐	deploy-expert-1775747728	Europe (Paris) eu-west-3	09 Apr 2026 05:15:30 PM CEST
☐	mon-projet-final-monbucketdonnees-pjktcdahlvvs	Europe (Paris) eu-west-3	09 Apr 2026 05:18:09 PM CEST
☐	monarchitectureexpert-monbucketexpert-nch5hl0zkmnc	Europe (Paris) eu-west-3	07 Apr 2026 02:39:41 PM CEST

Ressources

Actions d'API

Déployer l'API

Créer une ressource

/

ANY

Détails des ressources

Chemin

/

ID de ressource

lhqjxwri4

Méthodes (1)

Supprimer

Créer une méthode

	Type de méthode	Type d'intégration	Autorisation	Clé d'API
☐	ANY	Lambda	Aucun(e)	Facultatif

Conclusion et perspectives personnelles

Le projet mené au sein de l'entreprise Digit All Things a permis de démontrer avec succès la viabilité et l'intérêt stratégique d'une architecture hybride. En combinant la rapidité de développement front-end de Bubble.io avec la robustesse industrielle du back-end AWS, nous avons prouvé qu'il est possible de concilier agilité métier et stabilité technique. L'approche Serverless a parfaitement répondu aux exigences initiales de performance et de traçabilité, tout en garantissant une maîtrise totale des coûts opérationnels, un atout majeur pour la structure d'accueil.

Acquis et compétences développées

Ce stage a été un catalyseur pour mon développement professionnel, me permettant de transformer des connaissances théoriques en une expertise pratique sur plusieurs piliers du Cloud Computing :

- **Maîtrise avancée de Python** : Au-delà de la syntaxe, j'ai appris à utiliser Python comme un outil d'orchestration via le SDK Boto3, gérant des flux de données complexes entre API et bases de données.
- **Ingénierie Serverless** : J'ai acquis une compréhension profonde du cycle de vie des fonctions Lambda, de la gestion des déclencheurs et de l'optimisation des ressources sans serveur.
- **Culture DevOps et IaC** : La rédaction et l'utilisation de l'Infrastructure as Code (SAM/YAML) ont marqué un tournant dans ma méthode de travail. J'ai compris l'importance de la reproductibilité et de l'automatisation pour éliminer l'erreur humaine.
- **Sécurité Cloud** : L'application des principes du moindre privilège et des bonnes pratiques de sécurité AWS (chiffrement S3, rôles IAM granulaires) est désormais un réflexe de conception.

Perspectives d'évolution

Bien que la solution actuelle soit robuste et opérationnelle, le projet ouvre la voie à des optimisations de niveau "Enterprise Grade" :

Tout d'abord au niveau de la Sécurité Périmétrique, Pour qu'une application exposée à une plus large échelle, l'intégration d'AWS WAF (Web Application Firewall) constituerait la prochaine étape logique pour filtrer le trafic malveillant et prévenir les attaques de type injection ou DDoS. ensuite pour la Transition vers AWS CDK : c'est une évolution majeure consisterait à migrer l'Infrastructure as Code vers AWS CDK. Cela permettrait de définir l'infrastructure directement en Python, offrant un contrôle plus fin sur les ressources et une maintenabilité accrue pour les développeurs. Enfin la Gouvernance des données : La mise en place de politiques de cycle de vie sur S3 pour archiver automatiquement les données anciennes vers des classes de stockage moins coûteuses (comme S3 Glacier) et l'ajout de politiques IAM encore plus restrictives renforcerait la conformité et l'optimisation financière du projet.

En conclusion, cette expérience m'a permis d'appréhender le métier de Développeur Cloud / DevOps dans sa globalité, de la relation client à la mise en production, me donnant ainsi les clés pour aborder des architectures encore plus complexes à l'avenir.

Webographie

Vous retrouver ci-dessous les documents que j'ai pu utiliser durant ma formation afin de maîtriser toute la panoplie

configurer un environnement bubble et aws tout en faisant le lien entre eux :

<https://www.youtube.com/watch?v=rx7wsXqMUo0&t=3252s>

la création de permissions et la configuration :

<https://www.youtube.com/watch?v=HA59eSImpaw&t=134s>

information sur aws :

<https://www.youtube.com/watch?v=l0RlaQu3gWM&t=169s>

formation bubble :

https://www.youtube.com/watch?v=wi_wgBcbAJU&t=883s

<https://www.youtube.com/watch?v=xURF3CgVq1g&t=10s>

connection pour l'api de bubble

<https://www.youtube.com/watch?v=B7lgX0zaYlg&t=786s>

Dictionnaire de mots techniques

Cloud Computing (ou Cloud): C'est l'utilisation de serveurs informatiques à distance par Internet pour stocker des données ou exécuter des programmes, plutôt que de le faire sur son propre ordinateur.

· SaaS (Software as a Service): Un modèle où l'on utilise un logiciel directement en ligne via un abonnement, sans avoir besoin de l'installer (comme Bubble).

· No-Code: Une manière de créer des applications ou des sites web sans écrire une seule ligne de code informatique, en utilisant des interfaces visuelles.

· Full Stack Cloud: Désigne un développeur ou une solution capable de gérer à la fois la partie visible (le front-end) et la partie cachée/serveur (le back-end) dans un environnement cloud.

Serverless: Un mode de fonctionnement où l'utilisateur n'a pas à gérer les serveurs

informatiques; ils s'activent et se désactivent automatiquement selon les besoins.

AWS (Amazon Web Services): La plateforme de services cloud d'Amazon qui fournit

tous les outils nécessaires (stockage, calcul, base de données) pour faire fonctionner des

applications.

- Bubble: Une plateforme "no-code" permettant de construire l'interface visuelle d'une

application.

- Lambda: Un service d'AWS qui permet d'exécuter du code informatique (des "fonctions") uniquement quand c'est nécessaire, sans serveur permanent.

- DynamoDB: Une base de données très rapide proposée par AWS pour stocker des informations de manière organisée.

- S3 (Simple Storage Service) : Un service de stockage d'AWS utilisé pour conserver des fichiers (images, documents, archives) comme un disque dur géant sur Internet.

API (Application Programming Interface): Un "pont" qui permet à deux applications différentes (comme Bubble et AWS) de communiquer et d'échanger des données entre elles.

- JSON: Un format de texte standardisé utilisé pour structurer et échanger des données informatiques de manière simple.

- IAM (Identity and Access Management): Le système qui gère qui a le droit d'accéder à quoi dans l'environnement AWS.

- Chiffrement (ou TLS): Un procédé qui transforme les données en un code secret pour que personne ne puisse les lire pendant qu'elles circulent sur Internet.

- Zéro Trust (ou Moindre Privilège): Un principe de sécurité consistant à ne donner à un utilisateur ou à un programme que le strict minimum de droits dont il a besoin pour travailler,

DevOps: Une méthode de travail qui allie le développement de logiciels et la gestion des serveurs pour rendre les mises à jour plus rapides et fiables.

- Infrastructure as Code (IaC): Le fait d'utiliser des fichiers de texte (comme le format YAML) pour configurer automatiquement toute une infrastructure informatique au lieu de le faire manuellement.

- CI/CD (Continuous Integration/Continuous Deployment): Une chaîne automatisée qui teste et déploie les modifications d'une application dès qu'elles sont prêtes.

- SDK (Software Development Kit) :Un ensemble d'outils et de bibliothèques (comme Boto3 pour Python) qui aide les développeurs à programmer des interactions avec un service spécifique.
- Rollback: Une opération qui permet de revenir instantanément à une version précédente qui fonctionnait si une mise à jour rencontre un problème.